

BERECHENBARKEIT UND KOMPLEXITÄT

Übungsblatt 2

Prof. Rossmanith
C. Grüne, T. Janßen, T. Koglin
Lehrstuhl für Informatik 1
RWTH Aachen

WS 24/25
22.10.2024
Abgabe: Di. 29.10., 16:30

Lösung

Tutoraufgabe 2.1

Beschreiben Sie eine 1-Band-TM (in Worten), die die Sprache $L = \{0^n 1^n \mid n \in \mathbb{N}\}$ mit einem Zeitbedarf von $\mathcal{O}(m \log m)$ entscheidet, wobei m die Länge der Eingabe bezeichnet.

Lösung:

Aufgabe: Akzeptiere $L = \{0^n 1^n \mid n \in \mathbb{N}\}$ in Zeit $\mathcal{O}(m \log m)$.

Idee 1: Teile iterativ die Anzahl der Einsen und Nullen durch zwei und überprüfe jeweils, ob der Rest übereinstimmt.

Der Wert m bezeichnet die Länge der Eingabe w .

- 1) Teste in einem Durchlauf, ob die Eingabe die Form $0^* 1^*$ besitzt. ($\mathcal{O}(m)$)
- 2) Markiere die Positionen rechts und links von w mit $\$$. ($\mathcal{O}(m)$)
- 3) Durchlaufe die Eingabe von links nach rechts und ersetze jede zweite '0' durch '#'. Zähle dabei modulo zwei (mit Hilfe verschiedener Zustände). ($\mathcal{O}(m)$)
- 4) Überschreibe anschließend jede zweite '1' durch '#'. Zähle dabei modulo zwei. Wähle die Zustände so, dass das Ergebnis von Schritt 3 nicht verloren geht. ($\mathcal{O}(m)$)
- 5) Vergleich die beiden Ergebnisse. Bei Gleichheit gehe zu Schritt 3, andernfalls verwirf die Eingabe.

In jedem Schritt halbiert sich die Anzahl von '1'-en und '0'-en mindestens. Daher wird die Schleife höchstens $\mathcal{O}(\log m)$ mal durchlaufen wird, und es ergibt sich ein Zeitbedarf von

$$\mathcal{O}(m) + \log m \mathcal{O}(m) = \mathcal{O}(m \log m).$$

Idee 2: Zähle die Anzahl der Einsen und die Anzahl der Nullen und vergleiche die Zähler. Wir verwenden hier drei Spuren: Auf der ersten steht die Eingabe, auf der zweiten der Zähler für die Nullen und auf der dritten der Zähler für die Einsen.

- 1) Teste in einem Durchlauf, ob die Eingabe die Form $0^* 1^*$ besitzt. ($\mathcal{O}(m)$)
- 2) Erzeuge in einem Durchlauf drei Spuren, ersetze also jedes Symbol $x \in \{0, 1\}$ der Eingabe durch $\begin{pmatrix} x \\ B \\ B \end{pmatrix}$. (Zeitbedarf: $\mathcal{O}(m)$)
- 3) Durchlaufe die Eingabe von links nach rechts bis zur ersten Eins und zähle dabei binär die Anzahl der Nullen auf der zweiten Spur. Dabei wird nach jedem Schritt der Zähler um eins verschoben, sodass sich das letzte Bit des Zählers auf der Position der aktuell gelesenen Null befindet (Zeitbedarf da m binär kodiert $\log m$ lang ist: $\mathcal{O}(m \log m)$)
- 4) Durchlaufe die Eingabe weiter von links nach rechts und zähle dabei binär die Anzahl der Einsen auf der dritten Spur. Auch hier wird der Zähler nach jeder gelesenen Eins nach rechts verschoben (Zeitbedarf: $\mathcal{O}(m \log m)$)
- 5) Der Zähler auf der zweiten Spur endet auf der Position der letzten Null auf der ersten Spur. Der Zähler auf der dritten Spur endet auf der Position der letzten Eins auf der ersten Spur. Die Zähler sind also maximal m Positionen voneinander entfernt. Verschiebe den Zähler auf der zweiten Spur soweit nach rechts, dass nun beide Zähler rechtsbündig übereinander stehen. (Zeitbedarf: $\mathcal{O}(m \log m)$)

- 6) Vergleiche die beiden Zähler und akzeptiere, wenn sie gleich sind, ansonsten verwerfe die Eingabe. ($\mathcal{O}(\log m)$)

Insgesamt ergibt sich also ein Zeitbedarf von

$$\mathcal{O}(m) + \mathcal{O}(m \log m) + \mathcal{O}(\log m) = \mathcal{O}(m \log m).$$

Alternativ könnte man in Schritt 4 auch den Zähler auf der zweiten Spur sukzessive herunterzählen. Man würde somit mit 2 Spuren auskommen und das Verschieben des Zählers in Schritt 5 wäre überflüssig. In Schritt 6 bliebe dann noch zu testen, ob der Zähler den Wert 0 besitzt. (Sollte der Zähler zwischenzeitlich schon negativ werden, kann auch schon in Schritt 4 abgebrochen werden.)

Tutoraufgabe 2.2

Für ein Wort $w = w_1 w_2 \dots w_n$ mit $w_i \in \{0, 1\}$ sei $|w| = n$ die Anzahl der in w vorhandenen Bits. Sei $L = \{w \# \text{bin}(|w|) \mid w \in \Sigma^*\}$ über dem Alphabet $\Sigma = \{0, 1\}$.

Beschreiben Sie eine 1-Band-TM, die L entscheidet. Analysieren Sie den Zeit- sowie den Speicherplatzbedarf der von Ihnen entworfenen Maschine.

Lösung:

Es wird für jedes in w gelesene Bit die im zweiten Teil der Eingabe binärkodierte Anzahl der Bits in w dekrementiert. Da nur ein Band zur Verfügung steht, muss die TM dabei für jedes w_i einmal über die komplette Eingabe laufen. Das Dekrementieren in den Schritten 2 und 3 funktioniert analog zur TM in Tutaufgabe 1.3.

1. Im Zustand q_0 : Falls (links-)erstes Eingabesymbol $\#$ ist, prüfe ob rechts von $\#$ nur 0-Bits stehen. Falls ja, terminiere mit JA, ansonsten mit NEIN. Falls (links-)erstes Eingabesymbol 0, 1 ist, ersetze dieses durch B, wechsel in Zustand q_1 , und laufe bis $\#$.
2. Wechsel in Zustand q_2 und laufe von $\#$ nach rechts bis zum ersten 1-Bit. Falls kein 1-Bit vorhanden ist, gib NEIN aus. Ansonsten wechsel bei 1 in Zustand q_3 und gehe bis zum (rechts-)ersten B.
3. Wechsel in Zustand q_4 und bewege Kopf nach links zurück. Solange keine 1 gelesen wird, bleibe in q_4 und ersetze 0-Bits durch 1-Bits. Wird ein 1-Bit gelesen, wird dieses durch ein 0-Bit ersetzt und die TM wechselt in Zustand q_5 .
4. Wandere nach links zum Anfang der verbliebenen Eingabe, wechsel zu q_0 und gehe zu Schritt 1.

Daraus folgt eine Laufzeit von $\Theta(n^2)$ für den obigen Algorithmus. Da die TM nur auf den Speicherzellen der Eingabe (plus 2 Randzellen) arbeitet, ist der Speicherbedarf $\mathcal{O}(n)$.

Tutoraufgabe 2.3

- (a) Gegeben sei die Turingmaschine $M = (\{q_1, q_2, q_3\}, \{0, 1\}, \{0, 1, B\}, B, q_1, q_2, \delta)$ mit δ wie folgt:

	0	1	B
q_1	$(q_3, 1, N)$	$(q_1, 0, R)$	(q_2, B, L)
q_3	$(q_1, 0, L)$	$(q_3, 1, L)$	(q_1, B, R)

Bestimmen Sie die Gödelnummer $\langle M \rangle$ von M wie in der Vorlesung definiert.

Lösung:

Die Gödelnummer $\langle M \rangle$ der Turingmaschine M hat die Form

$$\langle M \rangle = 111 \text{code}(1) 11 \text{code}(2) 11 \dots 11 \text{code}(6) 111$$

wobei $\text{code}(t)$ die Kodierung des t -ten Übergangs der Funktion δ ist.

Gemäß Vorlesung wird das Bandalphabet $\Gamma = \{0, 1, B\}$ von $\langle M \rangle$ durchnummeriert mit $X_1 = 0, X_2 = 1$ und $X_3 = B$. Die möglichen Kopfbewegungen werden benannt mit $D_1 = L, D_2 = N, D_3 = R$. Der t -te Übergang $\delta(q_i, X_j) = (q_k, x_l, D_m)$ wird kodiert durch den Binärstring $\text{code}(t) = 0^i 10^j 10^k 10^l 10^m$. Es

ergibt sich folgende Kodierung von δ :

$$\text{code}(1) = 0^1 10^1 10^3 10^2 10^2$$

$$\text{code}(2) = 0^1 10^2 10^1 10^1 10^3$$

$$\text{code}(3) = 0^1 10^3 10^2 10^3 10^1$$

$$\text{code}(4) = 0^3 10^1 10^1 10^1 10^1$$

$$\text{code}(5) = 0^3 10^2 10^3 10^2 10^1$$

$$\text{code}(6) = 0^3 10^3 10^1 10^3 10^3$$

Es gilt also:

$$\begin{aligned}\langle M \rangle &= 111 \text{code}(1) 11 \text{code}(2) 11 \dots 11 \text{code}(6) 111 \\ &= 1110^1 10^1 10^3 10^2 10^2 110^1 10^2 10^1 10^1 10^3 110^1 10^3 10^2 10^3 10^1 110^3 10^1 10^1 10^1 10^1 11 \\ &\quad 0^3 10^2 10^3 10^2 10^1 110^3 10^3 10^1 10^3 10^3 111\end{aligned}$$

- (b) Erinnern Sie sich daran, dass eine Gödelnummer eine eindeutige präfixfreie Kodierung ist, die einer Turingmaschine M ein Wort $\langle M \rangle$ über dem Alphabet $\{0, 1\}$ zuordnet. Die Kodierung des t -ten Übergangs bezeichnen wir mit $\text{code}(t)$. Wir betrachten eine TM M mit s Übergängen.

Welche der folgenden Kodierungen erfüllt die Eigenschaften einer Gödelnummer?

- (i) • $\delta(q_i, X_j) = (q_k, X_\ell, D_m)$ kodiert als $1^i 01^j 01^k 01^\ell 01^m$.
• $\langle M \rangle = 000 \text{code}(1) 00 \text{code}(2) 00 \dots 00 \text{code}(s) 000$.
- (ii) • $\delta(q_i, X_j) = (q_k, X_\ell, D_m)$ kodiert als $\text{bin}(i)1 \text{bin}(j)1 \text{bin}(k)1 \text{bin}(\ell)1 \text{bin}(m)$.
• $\langle M \rangle = 111 \text{code}(1) 11 \text{code}(2) 11 \dots 11 \text{code}(s) 111$.
- (iii) • $\delta(q_i, X_j) = (q_k, X_\ell, D_m)$ kodiert als $0^i 10^j 10^k 10^\ell 10^m$.
• $\langle M \rangle = 11 \text{code}(1) 1 \text{code}(2) 1 \dots 1 \text{code}(s) 1$.

Lösung:

Bei dieser Aufgabe geht es darum zu verstehen, warum die in der VL gewählte Gödelnummer so gewählt ist wie beschrieben. Hier nicht nur diskutieren welche Kodierung eine Gödelnummer ist, sondern auch warum nicht und wie man das Problem beheben könnte.

- (i) Ja, bei der hier angegebenen Kodierung handelt es sich um die Kodierung aus der Vorlesung bei der nur 0 und 1 vertauscht wurden.
- (ii) Nein, weil die Kodierung weder eindeutig noch präfixfrei ist. Durch die Verwendung der Binärdarstellung kann man nicht mehr nachvollziehen wo eine Kodierung eines Übergangs beginnt oder aufhört. Man kann die Kodierung eindeutig machen, indem man Techniken zur Kodierung von Binärzahlen unbekannter Länge verwendet, man kann z.B. eine Zahl i durch $0^{|\text{bin}(i)|} 1 \text{bin}(i)$ kodieren, also vor der Kodierung der Zahl in Unärkodierung die Länge der Zahl angeben. Wenn man die korrekte Kodierung verwendet, kann man auch präfixfreiheit zeigen und die Kodierung wird insbesondere für große Turingmaschinen kürzer. Allerdings ist eine solche Kodierung aufwendiger zu beschreiben und beweisen. Da die Länge der Kodierung meistens nicht relevant ist (sie ist konstant für eine feste TM), wird dieser Weg nicht gewählt.

Für die oben angegebene TM würde man folgende Kodierung erhalten, die man nicht mehr eindeutig dekodieren kann:

$$\begin{aligned}\langle M \rangle &= \textcolor{red}{111} 1 1 1 1 11 1 10 1 10 \textcolor{red}{11} 1 1 10 1 1 1 1 11 \textcolor{red}{11} 1 1 11 1 10 1 11 1 1 \textcolor{red}{11} \\ &\quad 11 1 1 1 1 1 1 1 1 \textcolor{red}{11} 11 1 10 1 11 1 10 1 1 \textcolor{red}{11} 11 1 11 1 1 1 1 1 1 \textcolor{red}{111}\end{aligned}$$

- (iii) Nein, weil die Kodierung nicht präfixfrei ist. Das Problem ist hier nur die fehlende Markierung des Ende, die Trennung der einzelnen Codes kann man trotzdem finden. Ohne die Markierung des Ende kann man beim Einlesen der Kodierung nie wissen, ob noch weitere Übergänge folgen oder wo ein neuer Teil der Eingabe, wie z.B. das Eingabewort der zu simulierenden TM beginnt.

Das Verfahren aus VL (eine 1 innerhalb der Codes und zwei 1en zur Trennung der Codes) dient nur der menschlichen Lesbarkeit. Wichtig ist nur die klare Unterscheidung der Ende Markierung.